

GraphTrip 서비스 프로토타입

사용자 행동 및 키워드 기반 개인화 여행 코스 추천 서비스

팀명 **Graph Navigators**

팀원 이수광 · 박종빈 · 심찬기 · 최정현

Python

Streamlit

Graph

Recommendation

좋아요 데이터, 키워드 분석, 개인화 그래프 추천, 이동 경로 최적화를 하나의 추천 파이프라인으로 연결했습니다.

행동 데이터

좋아요 / 저장

키워드

TF-IDF
K-Means

개인화

RWR
PageRank

코스

Dijkstra
Greedy

프로젝트 설명서 구성

구성 방향

문제 정의, 데이터, 알고리즘, 구현 방식, 결과, PMI 순서로 프로젝트 내용을 정리했습니다.

핵심 강조

UI 기능 설명은 줄이고, 자료구조와 알고리즘이 어느 단계에서 왜 쓰였는지에 집중합니다.

제출 항목

프로토타입 제목, 설명, 업무 분장, 입력 데이터, 알고리즘, 구현 방식, 결과 화면, PMI를 포함했습니다.

항목	포함 내용	슬라이드
①	프로토타입 제목, 팀명, 팀원	1
②	What / Who / When / Why	3
③	업무 분장과 담당 알고리즘	6
④	Input 데이터 모양과 출처	4, 15
⑤	자료구조와 알고리즘 사용 위치 및 이유	6~10
⑥	Architecture, 동작 방식, 핵심 코드 흐름	5, 11
⑦	최종 실행 결과 화면	12
⑧	Lessons learned PMI	13
추가	프로토타입 한계점과 보완 방향	14

GraphTrip은 인기순이 아니라 취향과 동선을 함께 봅니다

What: 무엇을 하는가

사용자의 좋아요/저장 데이터와 키워드 선호를 기반으로 여행 장소를 추천하고, 추천 장소를 이동 거리 기준의 하나의 코스로 정렬합니다.

Who: 누구를 위한 서비스인가

여행 계획을 쉽게 세우고 싶은 사용자, 처음 방문하는 지역에서 개인 취향에 맞는 장소와 효율적인 동선을 원하는 사용자입니다.

When: 언제 사용하는가

여행을 계획할 때, 주말 나들이 코스를 찾을 때, 특정 장소 방문 후 비슷한 분위기의 장소를 찾고 싶을 때 사용합니다.

Why: 왜 필요한가

기존 서비스는 인기순/지역순 추천이 많아 개인 취향을 충분히 반영하지 못합니다. GraphTrip은 행동 데이터, 키워드, 그래프 추천, 거리 최적화를 함께 활용합니다.

모든 모듈은 장소 ID를 공통 키로 연결됩니다

places.json

장소 ID, 장소명, 위도(lat), 경도(lng), 키워드 리스트를 저장합니다. 키워드 분석, 개인화 추천, 경로 최적화가 모두 이 데이터를 사용합니다.

users.json

사용자 ID와 선호 키워드를 저장합니다. Topic-Sensitive Ranking에서 특정 사용자의 키워드 가중치를 높이는 데 사용합니다.

user_likes.csv

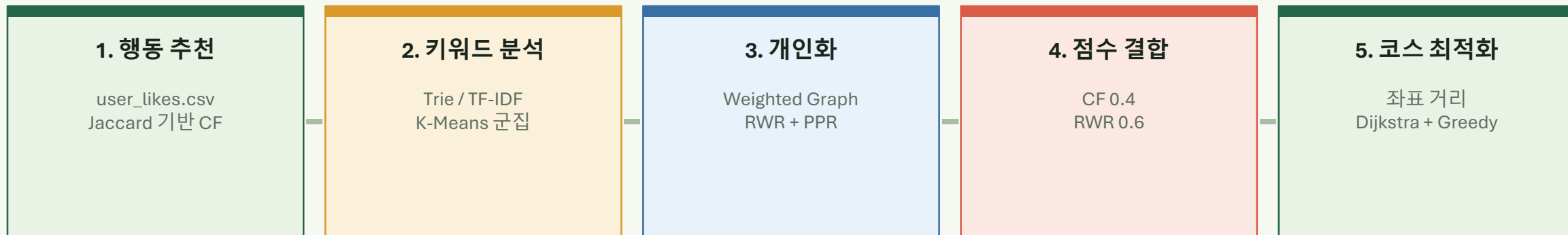
user_id, place_id 형식의 좋아요 데이터입니다. CF에서 사용자별 좋아요 집합을 만들고 유사 사용자를 찾는 입력입니다.

```
places.json 예시
{
  "id": 1,
  "name": "항리단길",
  "lat": 35.8354,
  "lng": 129.2104,
  "keywords": ["감성적인", "사진맛집", "데이트"]
}
```

데이터	프로토타입 내 사용 위치	출처
places.json	키워드, PPR, 경로 계산	팀 자체 구성
users.json	사용자 선호 키워드	더미 사용자 데이터
user_likes.csv	CF 유사도 계산	더미 행동 데이터

데이터 출처: 경주 주요 여행지를 기준으로 팀에서 직접 구성한 프로토타입용 데이터

추천은 네 개의 알고리즘 단계를 지나 최종 코스가 됩니다



통합 기준

모듈 간에는 장소 ID 리스트가 핵심 인터페이스입니다. 추천 모듈은 place_id를 만들고, 경로 모듈은 그 ID들을 방문 순서로 바꿉니다.

키워드 연결

TF-IDF와 K-Means로 장소의 키워드 성향을 분류하고, 키워드 가중치는 개인화 추천 그래프의 관계 계산에 활용됩니다.

결과 형태

추천 장소명, 추천 이유, 점수, 매칭 키워드, 최종 경로, 총 거리, 개선율을 함께 출력합니다.

핵심 구조: 추천 점수 생성 → 개인화 후보 정렬 → 거리 기준 방문 순서 최적화

업무 분장: 추천 결과가 코스까지 이어지는 구조

이수광 · CF 추천

사용자별 좋아요 집합을 만들고 비슷한 사용자의 장소를 후보로 추천

Set / Dictionary / Heap · CF / Jaccard

출력: CF 추천 place_id와 점수

박종빈 · 키워드 분석

장소 키워드를 벡터화하고 군집화해 사용자 키워드 가중치를 계산

Trie / HashMap / Matrix · TF-IDF / K-Means

출력: 키워드 가중치와 장소 군집

심찬기 · 개인화 추천

CF 후보와 키워드 가중치를 결합해 그래프 기반 개인화 점수 계산

Weighted Graph / Dictionary · RWR / PPR

출력: 최종 추천 장소 목록

최정현 · 경로 최적화

추천 장소를 좌표 기반 거리 그래프로 바꾸고 실제 방문 순서로 재배치

Graph / Priority Queue / Set · Dijkstra / Greedy

출력: 최적 여행 코스와 개선율

연결 기준

각 모듈은 내부 알고리즘은 다르지만, 최종적으로 place_id와 score를 다음 단계로 넘깁니다.

추천 모듈은 후보 장소를 만들고, 경로 최적화 모듈은 그 후보를 실제 여행 동선으로 바꿉니다.

Collaborative Filtering은 비슷한 사용자의 좋아요를 추천 후보로 만듭니다

1. 사용자별 좋아요 집합

user_likes.csv를 읽어 {user_id: set(place_id)} 형태의 Dictionary로 변환합니다.

2. Jaccard Similarity

두 사용자의 좋아요 장소 집합에서 교집합 / 합집합 비율을 계산해 취향 유사도를 구합니다.

3. 후보 점수 누적

유사 사용자가 좋아했지만 현재 사용자는 좋아하지 않은 장소에 유사도 점수를 누적합니다.

$$J(A, B) = |A \cap B| / |A \cup B|$$

사용 자료구조: Set은 교집합/합집합/차집합 계산, Dictionary는 사용자별 좋아요 집합과 후보 점수 저장, Heap은 Top-K 추천 추출에 사용됩니다.

User1

테스트 대상 사용자

0.3

유사도 threshold

[6, 7, 3]

CF 추천 place_id

Heap

상위 K개 후보 추출

키워드 시스템은 자연어 키워드를 벡터와 군집으로 바꿉니다

Trie + HashMap

키워드를 접두사 구조로 저장하고, 각 키워드를 벡터 인덱스로 매핑합니다.

TF-IDF

장소별 키워드 리스트를 문서로 보고, 자주 등장하지 않는 특징 키워드에 더 큰 가중치를 부여합니다.

K-Means

TF-IDF 벡터의 유클리드 거리를 기준으로 유사한 키워드 성향의 장소를 3개 군집으로 분류합니다.

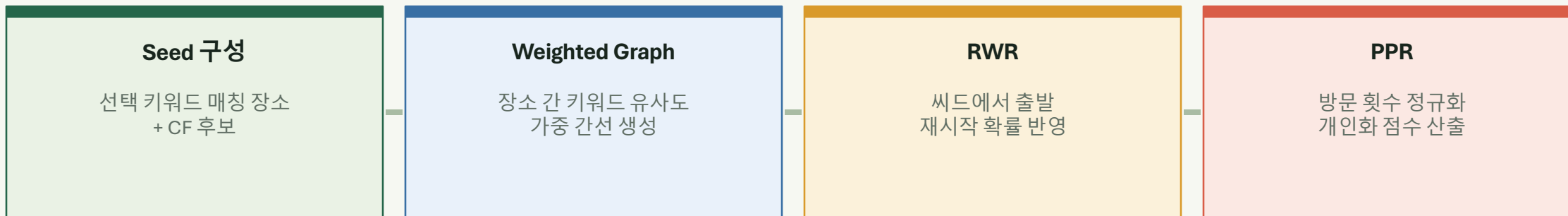
Topic-Sensitive

사용자 선호 키워드는 boost factor로 가중치를 높여 개인화 추천에 반영합니다.

군집	대표 키워드	분류된 장소
1	야경 · 데이트 · 산책	동궁과 월지, 경주월드, 보문호, 월정교
2	감성적인 · 데이트 · 한옥감성	황리단길, 교촌한옥마을
3	전통적인 · 고즈넉한 · 역사탐방	첨성대, 불국사, 석굴암, 국립경주박물관

K-Means 군집 결과는 장소를 키워드 성향별로 나누어 추천 설명과 탐색 화면에 활용합니다.

키워드와 CF 후보를 그래프 위에서 확산합니다



왜 그래프인가

장소를 노드로 두고, 키워드 유사도를 간선 가중치로 두면 사용자의 취향이 비슷한 장소 주변으로 추천 점수가 자연스럽게 퍼집니다.

추천 장소	점수	추천 이유
보문호	0.2308	야경 키워드 기반
첨성대	0.2258	CF 후보 기반
석굴암	0.2035	연관 장소 추천

사용 자료구조: Weighted Graph는 장소 간 관계를 저장하고, Dictionary는 점수와 방문 횟수, 추천 이유를 관리합니다.

경로 최적화는 추천 목록을 실제 방문 순서로 바꿉니다

좌표 로드

places.json
lat / lng

거리 계산

Haversine
직선거리

최단거리

Dijkstra
Priority Queue

방문 순서

Greedy
가까운 미방문 선택

핵심 아이디어

현재 위치에서 아직 방문하지 않은 추천 장소들까지의 최단 거리를 계산하고, 가장 가까운 장소를 다음 방문지로 선택합니다.

한계와 보완

Greedy는 빠르고 설명하기 쉽지만 전체 최적해를 항상 보장하지는 않습니다. 본 프로토타입에서는 시연성과 계산 효율을 우선했습니다.

32.32km

기존 추천 순서 거리

14.86km

최적화 후 거리

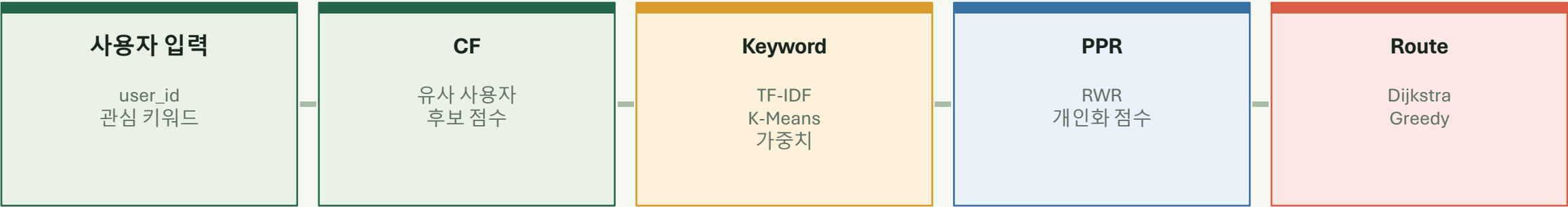
54.02%

개선율

place_id

추천 모듈과 연결 키

모듈은 place_id와 score를 기준으로 이어집니다



단계	입력	처리	출력
행동 추천	user_likes.csv	Jaccard + CF 점수	CF 후보 place_id
키워드 분석	places.json keywords	TF-IDF + K-Means	키워드 가중치
개인화 추천	키워드 씨드 + CF 후보	RWR + PPR 정규화	추천 장소 + score
경로 최적화	추천 place_id + 출발지	Dijkstra + Greedy	최종 여행 코스

점수 결합

RWR/PPR 점수 0.6과 CF 점수 0.4를 합산해 최종 추천 순위를 정합니다.

연동 기준

세부 알고리즘은 달라도 공통 key인 place_id를 통해 결과를 전달합니다

추천 장소는 최종적으로 거리 기준 여행 코스로 출력됩니다

 경로 추천 (역할4 Dijkstra)

출발지

알고리즘

항리단길

dijkstra

방문할 장소 선택 (경유지)

교촌한옥마을 × 동궁과 월지 × 월정교 ×

경로 최적화

 최적화된 여행 코스 (Dijkstra + Greedy)

항리단길 → 동궁과 월지 → 월정교 → 교촌한옥마을

최적 거리

기존 순서 거리

개선율

2.48km

2.98km

16.69%

구간별 이동 거리

항리단길 → 동궁과 월지: 0.62km

동궁과 월지 → 월정교: 1.15km

월정교 → 교촌한옥마을: 0.71km

코스 유형: 감성 데이트 코스

 경로 저장

시연 입력

출발지: 항리단길

방문 장소: 교촌한옥마을, 동궁과 월지, 월정교

최적 코스

항리단길 → 동궁과 월지 → 월정교 → 교촌한옥마을

2.48km

최적 거리

16.69%

개선율

2.98km

기존 거리

코스 유형: 감성 데이트 코스

결과 화면은 Streamlit 실행 화면이며, 경로 계산은 최신 route_optimizer.py의 좌표 기반 Dijkstra + Greedy 방식입니다.

한계점은 실제 데이터와 지도 기반 확장으로 보완합니다

데이터 규모

현재: 더미 사용자와 제한된 장소 데이터

보완: 실제 저장, 좋아요, 리뷰 로그 누적

신규 사용자 문제

현재: 좋아요 기록이 없으면 CF 적용이 어려움

보완: 초기 선호 키워드와 인기 장소 추천

이동 거리 계산

현재: 좌표 기반 직선거리라 실제 이동과 차이

보완: 지도 API로 실제 이동 시간 반영

경로 최적화

현재: Greedy는 항상 전체 최적을 보장하지 않음

보완: TSP, 2-opt, A*로 경로 품질 개선

최종 보완 방향

추천 품질은 실제 사용자 로그와 평가 지표로 검증하고, 경로 품질은 실제 이동 시간 기반 그래프로 개선합니다.

입력 데이터는 place_id를 공통 키로 연결됩니다

places.json

id, name, lat/lng, keywords 저장
모든 추천 모듈이 참조하는 장소 데이터

users.json

사용자 ID와 선호 키워드 저장
신규 사용자 추천의 초기 입력

user_likes.csv

user_id, place_id로 좋아요 기록 저장
CF에서 유사 사용자를 찾는 입력

```
places.json 예시
{
  "id": 1,
  "name": "황리단길",
  "lat": 35.8362,
  "lng": 129.2181,
  "keywords": ["감성적인", "사진맛집", "데이트"]
}
```

```
user_likes.csv 예시
user_id,place_id
1,1
1,2
1,8
1,10

recommended_place_ids = [1, 3, 10]
```

모듈별 사용 위치

CF 추천: user_likes.csv로 유사 사용자 계산
키워드 분석: keywords로 TF-IDF, K-Means 수행
개인화 추천: place_id별 추천 점수 계산
경로 최적화: lat/lng로 거리 그래프 생성

행동 데이터

user_likes.csv
좋아요 기록

키워드 데이터

places.json
장소별 키워드

개인화 점수

place_id + score
추천 후보 정렬

경로 입력

start_place_id
recommended_place_ids

데이터 출처: 경주 주요 여행지를 기준으로 팀에서 직접 구성한 프로토타입용 데이터

PMI로 팀원별 구현 경험을 정리했습니다

이수광 · CF 추천

- P** CF와 Jaccard로 행동 기반 추천 원리를 구현했습니다.
- M** 입출력 포맷과 평가 기준을 초반에 더 명확히 정하지 못했습니다.
- I** CF 결과가 PPR과 경로 최적화로 이어지는 흐름이 인상적이었습니다.

박종빈 · 키워드 분석

- P** TF-IDF와 K-Means로 키워드를 추천 가중치로 바꿨습니다.
- M** 부스팅 배율과 대용량 처리 방식을 충분히 검증하지 못했습니다.
- I** 자연어 키워드가 수치 점수로 바뀌는 과정이 흥미로웠습니다.

심찬기 · 개인화 추천

- P** PPR로 사용자 취향을 장소 그래프 위에서 확산시켰습니다.
- M** Cold Start와 역할 경계를 초반에 더 구체화하지 못했습니다.
- I** Random Walk와 PageRank의 연결 관계를 이해한 점이 흥미로웠습니다.

최정현 · 경로 최적화

- P** Dijkstra와 Greedy로 추천 장소를 실제 여행 코스로 바꿨습니다.
- M** Greedy와 직선거리 기반 계산은 실제 최적 경로와 차이가 있습니다.
- I** 방문 순서만 바뀌도 총 이동 거리가 달라지는 점이 인상적이었습니다.